

Architecture Diagram — Refinement

Architecture Diagram — Refinement

Current Backend Module State (Before)

Module	Models	Problem
module_project	ProfileType, Project, Pond, Cycle, CycleDailyHealth, CycleStageMetric	Too big — 6 models, mixing project config with pond data and growth cycles
module_sensor	ParameterType, SensorType, IoTDevice, Sensor (legacy), ProjectSensor, ProjectParameterSetting, SensorMessage, SensorReading	8 models — mixing config with data
module_data_ingestion	PondReadingDataTable + IngestionService, BroadcastService, ThresholdService	Partition tables are legacy, but services are actively used by MQTT pipeline
module_notification	Alert (legacy), AlertLog	Fine — 1 active model
module_user	User, Role (old)	Needs RBAC additions
module_chart	Services only (no models)	Missing entity ownership

Consultant Feedback

- Project module is too big. Not modularized.
- She does not like everything inside the project module, even in this stage.
- Need to revise module identification/boundaries.

Proposed Module Breakdown (After)

1. module_user — Authentication & Access Control

Models	Purpose
User	User profile and credentials
Role	Role definitions with module/feature permissions (JSONB)
UserRole	Junction: which roles a user has
UserRoleProject	Junction: which projects a user-role can access
ModuleAccess	Reference: valid module codes in the system
FeatureAccess	Reference: valid feature codes in the system

Justification: All authentication, authorization, and RBAC logic belongs here. This module handles: login flow, role selection, permission checking, and user profile management. It’s the gatekeeper — every API request passes through this module’s permission layer before reaching other modules. Keeping access control isolated makes it easier to audit, test, and evolve the permission model independently.

2. module_project — Project Setup & Configuration

Models	Purpose
ProfileType	Template definitions for aquaculture types (shrimp, fish, crab, treatment) — stage config, key indicators
Project	Project/farm entity — links owner, profile type, and serves as the top-level grouping

Justification: This module is about **setup and configuration**, not operational data. A project is created once, configured with a profile type, and rarely changes after that. By keeping only Project and ProfileType here, the module stays focused on “what is this farm and how is it configured?” — not “what’s happening in the ponds right now.” This is the admin/setup layer.

3. module_pond — Pond & Growth Cycle Management

Models	Purpose
Pond	Physical pond/tank entity with metadata, status, location
Cycle	Production cycle for a pond (start date, end date, status)
CycleDailyHealth	Daily health status record per day in a cycle (colored circles on timeline)
CycleStageMetric	Aggregated growth metrics per stage (JSONB with avg/min/max per indicator)

Justification: Pond is the **central operational entity** in the system. Sensors are installed in ponds. Readings come from ponds. Alerts are generated for ponds. Cycles run in ponds. Separating Pond from Project reflects the reality that most of the app’s features are pond-centric, not project-centric.

Cycles are included here because they are **tightly coupled to ponds** — every cycle belongs to a pond (`cycle.pond_id`), no cycle exists without a pond, and the Historical Data page queries pond + cycles together. Having a separate `module_cycle` with 3 models while `module_pond` only had 1 model would be over-modularized. Combining them into one module keeps the boundary clean:

module_project = farm setup/config, module_pond = operational data (ponds, cycles, health, metrics).

4. module_sensor — Sensor Configuration & Hardware Management

Models	Purpose
ParameterType	Reference: water quality parameters (temperature, pH, etc.) with code, name, unit
GrowthIndicator	Reference: growth performance metrics (body_weight, FCR, etc.) with code, name, unit
SensorType	Sensor hardware catalog (model, manufacturer, which parameters it measures)
IoTDevice	IoT gateway device (Raspberry Pi) — device code, status, auth key
ProjectSensor	Sensor assignment: links a sensor type + IoT device to a project + pond
ProjectParameterSetting	Per-project parameter thresholds (min/max) and key parameter flag

Justification: This is the **sensor configuration module** — it defines what sensors exist, what they can measure, which device they’re connected to, and where they’re installed. It’s the admin/setup side of the sensor domain. These entities are configured once and rarely change during operation. Separating configuration from data ingestion keeps this module focused on “what is set up” vs “what data is flowing.”

5. module_data_ingestion — Data Pipeline & Real-time Processing

Models	Purpose
SensorMessage	Raw MQTT message from IoT device (payload, sequence number, timestamps)
SensorReading	Parsed sensor reading with 22 parameter columns (the clean, queryable data)
Services	Purpose
IngestionService	Validates MQTT payload, dedup check, inserts SensorMessage + SensorReading
BroadcastService	Pushes readings + alerts to WebSocket channels (Django Channels)
ThresholdService	Compares readings against thresholds, creates/resolves AlertLog entries
mqtt_adapter	Management command — MQTT subscriber, validates signatures, calls IngestionService

Justification: Data ingestion is a **runtime process** — it’s always running, processing incoming MQTT messages in real-time. This is fundamentally different from sensor configuration (which is admin/setup). The ingestion pipeline handles: receive → validate → dedup → store → check thresholds → broadcast. It’s write-heavy and latency-sensitive. Keeping it separate means: - Ingestion logic can evolve independently (e.g., batch processing, data transformation rules, validation pipelines) - Configuration changes (module_sensor) don’t risk breaking the data pipeline - Clear separation of concerns: module_sensor = “what is set up”, module_data_ingestion = “what data is flowing” - The services (IngestionService, BroadcastService, ThresholdService) are already here and actively used by the MQTT pipeline

Cleanup needed: Remove legacy partition code (PondReadingDataTable model, partition_manager.py, old IngestConsumer). Keep only the active MQTT pipeline services.

6. module_chart — Visualization & Chart Configuration

Models	Purpose
VisualisationType	Chart type catalog (line, bar, heatmap, etc.) with required parameters
ProjectVisualisation	Per-project chart configuration (which charts enabled, x/y parameters, title)

Justification: This module owns **what charts exist and how they’re configured per project**. The chart rendering services already live here. Adding the entity models (VisualisationType, ProjectVisualisation) gives this module full ownership of the visualization domain — from chart definition to chart rendering. Without these entities, the module was just a service layer with no data ownership, which is an architectural smell.

7. module_notification — Alerts & Notifications

Models	Purpose
AlertLog	Alert records with acknowledge/resolve lifecycle

Justification: Alert management is a cross-cutting concern — alerts are triggered by sensor readings (module_data_ingestion), displayed on multiple pages (Overview, Real-time), and acknowledged/resolved by users (module_user). Having its own module keeps the alert lifecycle (create → acknowledge → resolve) cleanly separated. As notifications evolve (email alerts, SMS, push notifications, escalation rules), this module can grow independently without affecting sensor or ingestion logic.

Summary

#	Module	Models	Services	Focus
1	module_user	6	Auth, permissions	Auth, RBAC, permissions
2	module_project	2	—	Project setup & profile config
3	module_pond	4	—	Pond + growth cycle management
4	module_sensor	6	Config services	Sensor config, hardware, parameters
5	module_data_ingestion	2	IngestionService, BroadcastService, ThresholdService, mqtt_adapter	Data pipeline & real-time processing
6	module_chart	2	Chart rendering	Chart config & rendering
7	module_notification	1	—	Alerts & notifications
	Total	23		7 modules

Before vs After

Before	After
module_project had 6 models	Split into module_project (2) + module_pond (4 — Pond + Cycle + DailyHealth + StageMetric)
module_sensor had 8 models (config + data mixed)	Split into module_sensor (6 config) + module_data_ingestion (2 data)
module_chart had no entities	Now owns VisualisationType + ProjectVisualisation
module_data_ingestion was legacy partitions only	Repurposed — now owns data pipeline (SensorMessage, SensorReading, ingestion services)
module_user had 2 models (old RBAC)	Now has 6 models (full RBAC)

Module Dependencies

module_user ← (all modules check permissions)

module_project → module_user (project_owner_id)

module_pond → module_project (pond.project_id, cycle→pond→project)

module_sensor → module_project, module_pond (project_sensor links both)

module_data_ingestion → module_sensor (reads config), module_pond (pond_id), module_notification (creates alerts)

module_chart → module_project (project_visualisations), module_sensor (parameter references)

module_notification → module_pond, module_project (alert_log references both)

Last updated: April 26, 2026